

DLOps Lab Worksheet – CNN on CIFAR-10

Name: Kunal Mishra

Roll Number: M25CSA036

Course: MLOps / DLOps

Lab: CNN Training & Analysis

1. Objective:

The objective of this lab is to design and train a Convolutional Neural Network (CNN) on the CIFAR-10 dataset and analyze its training behavior from a DLOps perspective. The experiment focuses on building a custom data pipeline, measuring computational complexity using FLOPs, monitoring gradient flow and weight update flow, and logging all visualizations using Weights & Biases (WandB).

2. Dataset Description (CIFAR-10)

The CIFAR-10 dataset is a standard benchmark dataset used for image classification tasks.

- Total images: 60,000
- Training images: 50,000
- Test images: 10,000
- Image size: 32×32
- Channels: 3 (RGB)
- Number of classes: 10

A custom **PyTorch Dataset class** was implemented by inheriting from `torch.utils.data.Dataset`. The dataset applies tensor conversion and normalization to ensure stable and efficient training.

3. Model Architecture

A custom CNN architecture was designed for CIFAR-10 classification. The model consists of:

- Three convolutional layers with increasing channels
- ReLU activation functions
- Batch Normalization for training stability

- Max Pooling layers for spatial reduction
- Two fully connected layers for classification

4. Custom DataLoader

Instead of directly using the built-in CIFAR-10 dataset, a **custom DataLoader** was created. The custom dataset class overrides:

- `_init_()` for dataset initialization
- `_len_()` to return dataset size
- `_getitem_()` to return individual samples

This approach reflects real-world DLOps practices where data pipelines are customized for flexibility and control.

4. FLOPs Analysis

Floating Point Operations (FLOPs) were calculated to measure the computational complexity of the CNN model.

- Computed using `fvcore` library
- Input size: $3 \times 32 \times 32$
- Single forward pass FLOPs measured
- Model found to be computationally efficient

The analysis showed that the model is computationally efficient and suitable for CIFAR-10-scale tasks.

6. Training Setup

The model was trained with the following configuration:

- Optimizer: Adam
- Loss Function: Cross-Entropy Loss
- Batch Size: 64
- Epochs: 25
- Hardware: GPU (Google Colab)

Training progress was displayed using a live progress bar, while epoch-level matrices were logged to weights and biases.

7. Gradient Flow Visualization

Gradient flow analysis was performed to study how gradients propagate through different layers during backpropagation.

- Mean absolute gradients were computed for each trainable layer
- Gradient flow was visualized once per epoch
- Bias parameters were excluded for clarity

Observation:

Gradients remained stable across layers throughout training, with no signs of vanishing or exploding gradients. Early layers showed slightly smaller gradients compared to deeper layers, which is expected in CNNs.

8. Weight Update Flow Visualization

Weight update flow was analyzed by measuring how much model parameters changed during training.

- Model weights were saved before each optimizer step
- Weight update norms were computed after the update
- Updates were averaged across batches per epoch

Observation:

Weight updates were larger in early epochs and gradually decreased in later epochs, indicating stable convergence of the model.

9. Experiment Tracking using WandB

Weights and Biases (WandB) was used to log and visualize:

- Training loss
- Training accuracy
- Validation accuracy
- Gradient flow plots

- Weight update metrics

WandB provided real-time monitoring, reproducibility, and a centralized dashboard for analyzing training dynamics.

10. Results and Observations

- Training loss decreased consistently across epochs
- Training and validation accuracy improved and stabilized
- Gradient flow remained healthy throughout training
- Weight updates reduced over time, indicating convergence
- No major overfitting was observed

Overall, the training process was stable and well-monitored.

11. Conclusion

This lab successfully demonstrated a complete DLOps workflow for training a CNN on the CIFAR-10 dataset. By implementing a custom data pipeline, performing FLOPs analysis, monitoring gradient and weight dynamics, and using WandB for experiment tracking, the experiment emphasized both model performance and operational best practices in deep learning.

12. References

- CIFAR-10 Dataset
- PyTorch Documentation
- fvcare FLOPs Analysis
- Weights & Biases Documentation